PROJECT PROPOSAL · TECHNICAL GUIDE · ACCESS CONTROL

Starling Application Portal

Access Control Platform: Proposal and Technical Guide

Next.js · Supabase · ESP32-S3 door controllers

DOCUMENT	Proposal and technical implementation guide
VERSION	2.0
DATE	21 October 2026
AUTHOR	Mazwewoh John Brindi
STATUS	Ready for installation
AUDIENCE	Stakeholders, installers and developers
REPOSITORY	starling-application-portal

Prepared by Mazwewoh John Brindi. Confidential, for internal circulation.

CONTENTS

Table of Contents

1. Executive Summary 4
- At a Glance 4
- Why This Design 5

2. How the System Works 6

3. System Architecture 7
- Who Talks to Whom 8
- Key Architectural Decisions 8

4. Door Hardware 9
- Bill of Materials (per door) 10
- Pin Map (ESP32-S3) 11
- Lock Power Wiring 11
- Mounting and Physical Security 12

5. Door Controller Firmware 13
- Firmware Stack 13
- Libraries 13
- Firmware Modules 14
- Main Loop 14
- The Decision Logic 14
- Offline Mode 15
- Device Provisioning 15

6. Credentials 16
- QR Code on the Phone 16
- PIN 16
- Fingerprint 16
- Face Recognition 17
- RFID Cards (optional) 17
- Door Policies 17

7. Device API (Supabase Edge Functions) 18
- Request Signing 18
- Endpoints 18
- Rule Snapshot Example 19
- Remote Unlock 19

8. Web Application Modules 20

Public and Accounts 20
Access Requests (the application) 20
My Access (member area) 20
Approvals and Grants (admin) 20
Doors, Devices and Schedules (admin) 20
Live Monitoring (officer) 20
Reports and Audit 21
Users and Roles 21
Route Map 21

9. Technology Stack and Tools 22
Web Application 22
Cloud Backend: Supabase 22
Hardware and Firmware Tools 23
Development and Operations 23

10. Database Design 24
Enumerations 24
Tables 25
Triggers and Jobs 26

11. Security and Privacy 27
Cloud and Web 27
Devices 27
Physical 27
Personal and Biometric Data 27

12. Project Structure 28

13. Setup Guide 29
1. Supabase Project 29
2. QR Signing Keys (once) 29
3. Web Application 29
4. Firmware 30

14. Installation Day Guide 31
Before Going On Site 31
On Site 31
Acceptance Tests 32

15. Testing and Quality 33

16. Risks and Mitigations 34

17. Appendix 35

SECTION 01

1. Executive Summary

Starling is an access control platform. People apply for access in a web portal, an administrator approves them, and ESP32 door controllers then let them in using a QR code, a PIN or a fingerprint. Every entry is logged and visible live.

The platform has three parts that work together: a **Next.js web application** for people and administrators, a **Supabase cloud backend** that stores users, rules and logs, and an **ESP32-S3 door controller** at every door that reads credentials and drives the lock. Door controllers keep a local copy of the access rules, so doors keep working even when the internet is down.

This document explains exactly how the software talks to the hardware, lists every hardware part and how it is wired, defines the device API and the database, and gives step-by-step setup, installation and test procedures.

At a Glance

Item	Summary
Product	Starling Application Portal, a web-managed access control system.
How people get in	QR code on their phone, personal PIN on a keypad, or fingerprint. RFID cards are supported as an option.
Door hardware	ESP32-S3 controller, QR scanner, keypad, fingerprint sensor, relay, 12 V magnetic lock, exit button, door sensor, backup power.
Web application	Next.js (App Router), React, TypeScript, Tailwind CSS, shadcn/ui, hosted on Vercel.
Cloud backend	Supabase: Postgres with Row Level Security, Auth, Edge Functions (device API), Realtime, Storage, Vault.
Firmware	C++ on the Arduino framework for ESP32, built with PlatformIO.
Offline behaviour	Each door decides locally from cached rules and uploads its logs when the connection returns.
Author	Mazwewoh John Brindi

Why This Design

Doors never depend on the internet

The decision to open is made on the ESP32 from a local copy of the rules. The cloud is used to manage and record, not to open each door.

One web platform

Applications, approvals, devices, schedules, live logs and remote unlock all happen in one Next.js dashboard backed by Supabase.

Secure by design

QR codes are digitally signed and expire in about a minute, PINs are stored only as salted hashes, and fingerprint templates never leave the sensor.

Low cost, easy to extend

Standard, widely available parts. Adding a door means adding one more controller and registering it in the dashboard.

SECTION 02

2. How the System Works

The full journey from someone applying for access to the door opening, step by step.



Figure 1. Access flow from application to open door.

Step	Where	What happens technically
1. Apply	Web portal	The user signs up (Supabase Auth), fills in an access request (doors or site, reason, dates) and uploads an ID document to Supabase Storage. A row is created in **access_requests** with status pending.
2. Approve	Admin dashboard	An administrator reviews the request and approves it. A server action creates rows in **access_grants** (user, door, schedule, valid from and until) and bumps the **rules_version** of every affected door.
3. Credentials	Portal and door	The user sets a PIN (stored as a salted hash). The QR page in the portal starts showing a signed QR token. If fingerprint is needed, an officer starts enrolment and the user places a finger on the door's sensor.
4. Sync	Device API	Within seconds the door controller's heartbeat sees that its rules version changed and calls **device-sync**. It downloads the new rule snapshot and saves it to flash.
5. Present	At the door	The user shows the QR code to the scanner, types a PIN on the keypad, or places a finger on the sensor.
6. Decide	ESP32	The firmware identifies the user, checks the grant is active, the door is allowed, the current time is within the schedule and the credential is valid. No internet is needed.
7. Unlock	ESP32 and lock	On success the relay switches, the magnetic lock releases for the configured time (default 5 seconds), the LED turns green and the buzzer beeps. On failure it shows red and the reason.
8. Log	Device API and dashboard	The event is queued and uploaded through **device-events** into **access_events**. Supabase Realtime pushes it to the dashboard instantly.

SECTION 03

3. System Architecture

Four layers: people, the web application, the Supabase cloud and the door controllers. The door controller never talks to the database directly; it only uses a small, authenticated device API.

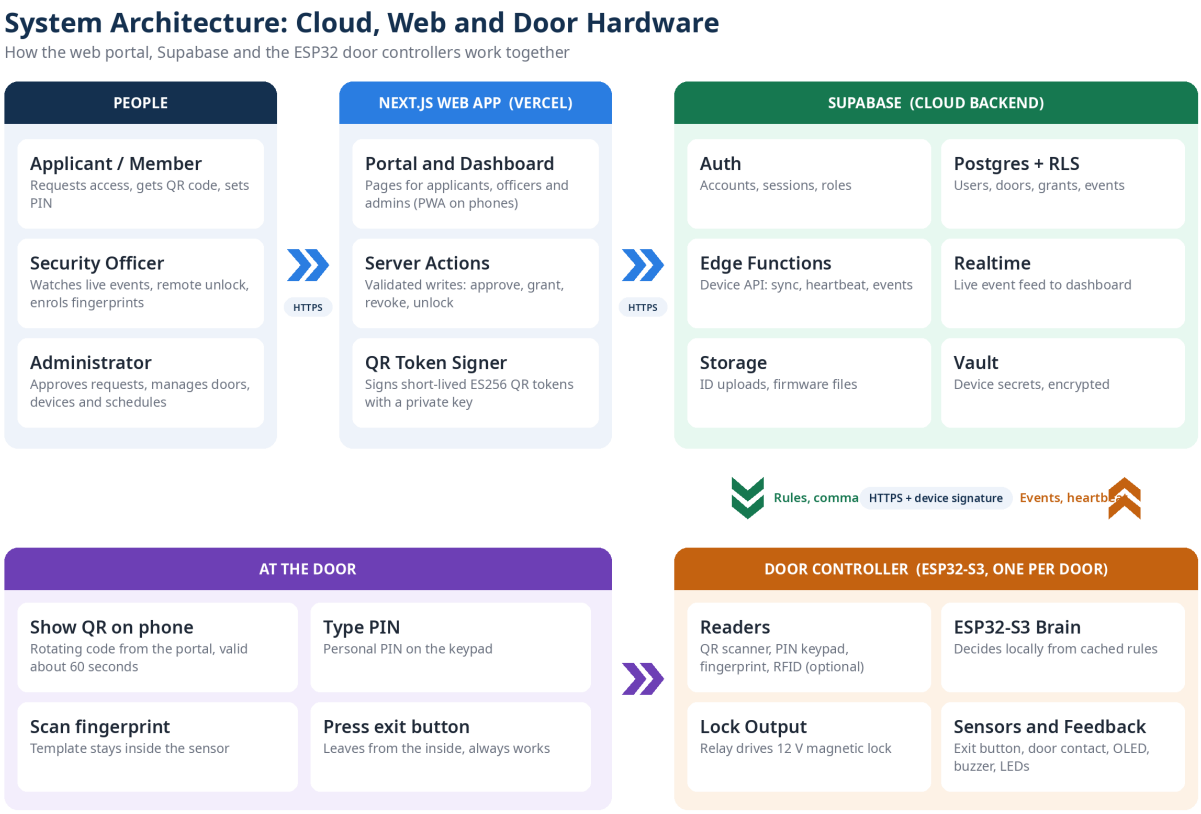


Figure 2. System architecture including cloud, web application and door hardware.

Who Talks to Whom

From	To	Protocol	Purpose	Authentication
Browser or phone	Next.js (Vercel)	HTTPS	Pages, forms, QR display	Supabase session cookie
Next.js server	Supabase Postgres	HTTPS (PostgREST)	Read and write data	User session, RLS applied
Dashboard browser	Supabase Realtime	WebSocket (WSS)	Live events and device status	User session, RLS applied
Door controller	Edge Function **device-heartbeat**	HTTPS POST every 5 s	Status, pending commands, rules version	Device ID + HMAC signature
Door controller	Edge Function **device-sync**	HTTPS POST on change	Download rule snapshot	Device ID + HMAC signature
Door controller	Edge Function **device-events**	HTTPS POST in batches	Upload access and alarm events	Device ID + HMAC signature
Door controller	Supabase Storage	HTTPS GET	Download firmware updates (OTA)	Short-lived signed URL
Phone screen	QR scanner	Optical	Present signed QR token	ECDSA signature in the token
Readers	ESP32	UART, I2C, SPI	Credential data	Physical wiring on secure side

Key Architectural Decisions

Decision	Rationale
Decide at the door, manage in the cloud	Doors open in under half a second and keep working through internet outages. The cloud is the source of truth for rules and the home of all logs.
Edge Functions as the device API	Devices never hold database keys. Each device has its own secret; a compromised device can be disabled in one click without affecting anything else.
Heartbeat polling for commands	Simple and reliable through firewalls. Remote unlock reaches the door in five seconds or less. MQTT can be added later for instant delivery.
Signed QR tokens	The door verifies the QR signature with a public key, so it can check QR codes offline. Tokens expire quickly, so screenshots cannot be reused.
Fingerprint templates stay on the sensor	Biometric data never travels over the network or sits in the cloud. The cloud only knows which sensor slot belongs to which user.
Server components and server actions	Secrets stay on the server, pages load fast and every write is validated in one place.

SECTION 04

4. Door Hardware

One controller per door. Readers are mounted on the outside; the ESP32, relay and power supply are mounted on the inside, so nobody can open the door by tampering with the reader wiring.

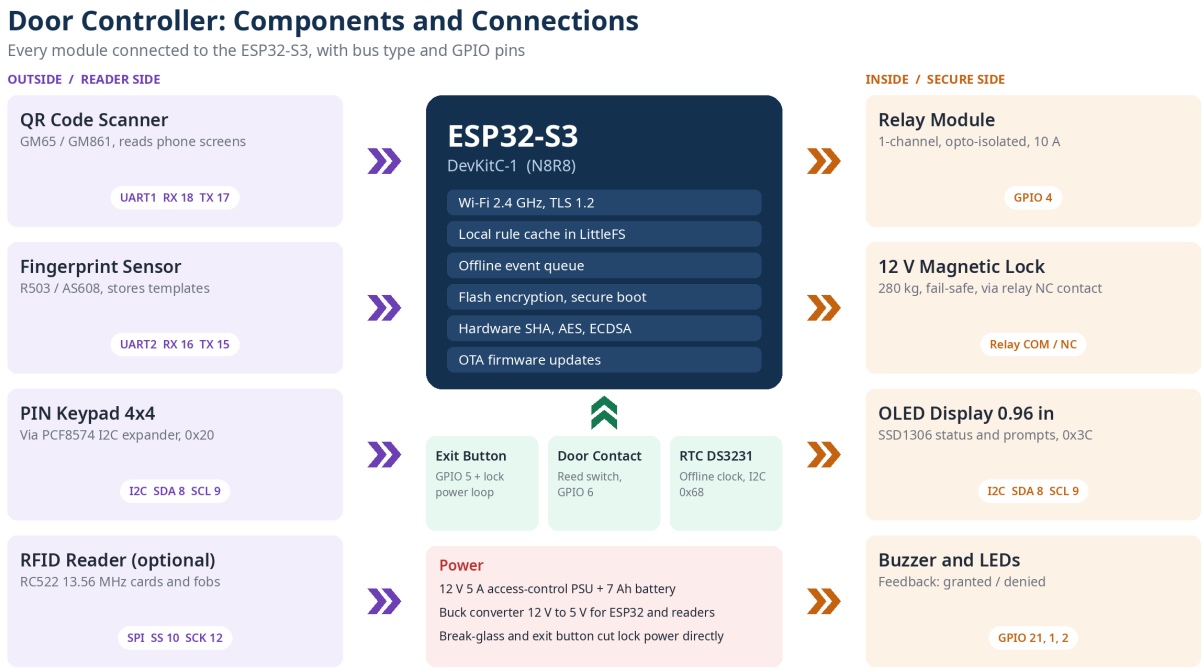


Figure 3. Door controller components, buses and GPIO pins.

Bill of Materials (per door)

Part	Recommended model	Qty	Role
Microcontroller	ESP32-S3-DevKitC-1 (N8R8: 8 MB flash, 8 MB PSRAM)	1	Runs the firmware, Wi-Fi, local decisions, logging.
QR code scanner	GM65 or GM861 UART module (screen-reading capable)	1	Reads the signed QR code from the user's phone screen.
PIN keypad	4x4 membrane or metal keypad + PCF8574 I2C expander	1	PIN entry, also used for menu and enrolment.
Fingerprint sensor	R503 (capacitive, ring LED) or AS608	1	Stores fingerprint templates and matches on the sensor.
RFID reader (optional)	RC522 13.56 MHz (Mifare) or PN532	0 to 1	Optional backup credential using cards or fobs.
Display	0.96 in SSD1306 OLED, I2C	1	Prompts and status: Scan QR, Enter PIN, Granted, Denied.
Real-time clock	DS3231 module with coin cell	1	Keeps accurate time offline for schedules and QR expiry.
Relay	1-channel 5 V opto-isolated relay, 10 A	1	Switches power to the lock.
Lock	12 V DC electromagnetic lock, 280 kg (600 lb), with bracket	1	Holds the door. Fail-safe: unlocks when power is cut.
Exit button	Request-to-exit button with NO and NC contacts	1	Lets people out from inside; NC contact cuts lock power directly.
Emergency release	Green break-glass unit	1	Manual emergency override in the lock power loop.
Door contact	Magnetic reed switch	1	Detects door open, forced entry and door held open.
Power supply	12 V 5 A access-control PSU with 12 V 7 Ah battery backup	1	Powers lock and electronics; keeps running during power cuts.
Voltage regulator	LM2596 or MP1584 buck converter, 12 V to 5 V, 3 A	1	Clean 5 V for ESP32, scanner and relay.
Feedback	Active buzzer, green and red LEDs with 220 ohm resistors	1 set	Audible and visual result.
Protection	1N4007 diode or MOV across lock, inline fuses	1 set	Suppresses lock coil spikes and protects wiring.
Enclosures and cable	IP65 reader housing, lockable controller box, 6-core alarm cable	1 set	Weather and tamper protection.
Optional wired network	W5500 Ethernet module or WT32-ETH01 board	0 to 1	Use where Wi-Fi is weak or unreliable.

Pin Map (ESP32-S3)

Module	Bus	Module pin	ESP32-S3 GPIO	Power
QR scanner GM65	UART1	TX / RX	RX 18 / TX 17	5 V, GND
Fingerprint R503	UART2	TX / RX / WAKEUP	RX 16 / TX 15 / IRQ 7	3.3 V, GND
Keypad via PCF8574	I2C 0x20	SDA / SCL	8 / 9	3.3 V, GND
OLED SSD1306	I2C 0x3C	SDA / SCL	8 / 9 (shared)	3.3 V, GND
RTC DS3231	I2C 0x68	SDA / SCL	8 / 9 (shared)	3.3 V, GND
RFID RC522 (optional)	SPI	SS / SCK / MOSI / MISO / RST	10 / 12 / 11 / 13 / 14	3.3 V only, GND
Relay IN	Digital out	IN	4	5 V, GND
Exit button (NO contact)	Digital in, pull-up	Signal	5	GND
Door contact	Digital in, pull-up	Signal	6	GND
Buzzer	Digital out	+	21	GND
Green LED / Red LED	Digital out	Anode via 220 ohm	1 / 2	GND

Pins to avoid on ESP32-S3

GPIO 0, 3, 45 and 46 are boot strapping pins, GPIO 19 and 20 are USB, and GPIO 35 to 37 are used by octal PSRAM on N8R8 boards. The pin map above avoids all of them. The RC522 is a 3.3 V device and must never be connected to 5 V.

Lock Power Wiring

- 12 V PSU positive goes through a fuse, then the **break-glass** NC contact, then the **exit button** NC contact, then the relay **COM** terminal.
- Relay **NC** terminal goes to the magnetic lock positive. Lock negative returns to PSU negative.
- Relay off (normal state): NC is closed, the lock is powered and the door is locked. Relay on: NC opens, lock power is cut and the door opens.
- If the ESP32 crashes or reboots, the relay is off and the door stays locked. If all power is lost, the magnetic lock releases (fail-safe), which most fire codes require on exit routes.
- Pressing the exit button or the break-glass breaks the lock power directly in hardware, so people can always leave even if the controller has failed.

- Fit a 1N4007 diode (or MOV) across the lock terminals, reverse-biased, to absorb the voltage spike when the coil switches off.
- The buck converter feeds 5 V to the ESP32 5V pin, the QR scanner and the relay module. All grounds are common.

Safety and compliance

A fail-safe magnetic lock must be used on any door that is an escape route. Where a fire alarm panel exists, wire its release contact into the same lock power loop so the door unlocks when the alarm activates. Have the final electrical work checked by a qualified electrician.

Mounting and Physical Security

- Outside: only the QR scanner, keypad, fingerprint sensor and optional RFID reader, in an IP65 housing.
- Inside, in a lockable box: ESP32, relay, buck converter, RTC and the PSU with battery.
- Run reader cables through the wall into the secure side; nothing that can switch the lock is reachable from outside.
- Add a tamper switch to the controller box lid (optional, any free GPIO) to raise an alert when it is opened.

SECTION 05

5. Door Controller Firmware

The firmware is the bridge between the hardware and the platform. It reads credentials, decides, drives the lock, keeps rules up to date and reports everything back.

Firmware Stack

Item	Choice
Language and framework	C++ on the Arduino core for ESP32 (built on ESP-IDF).
Build tool	PlatformIO in VS Code. Arduino IDE 2 also works for quick tests.
Storage	LittleFS for the rule snapshot and event queue; NVS (Preferences) for device ID, secret and settings.
Networking	WiFi + HTTPClient with WiFiClientSecure (TLS, Supabase root CA certificate pinned).
Cryptography	mbedTLS built into ESP32: HMAC-SHA256 for request signing, PBKDF2 for PINs, ECDSA P-256 for QR tokens.
Updates	HTTPS OTA (esp_https_ota) from a signed URL in Supabase Storage.

Libraries

Library	PlatformIO name	Used for
ArduinoJson	bblanchon/ArduinoJson	Parsing sync snapshots and building event payloads.
Adafruit Fingerprint	adafruit/Adafruit Fingerprint Sensor Library	Enrol, search and delete templates on R503 / AS608.
I2CKeyPad	robtillaart/I2CKeyPad	4x4 keypad through the PCF8574 expander.
U8g2	olikraus/U8g2	OLED text and icons.
RTClib	adafruit/RTClib	DS3231 real-time clock.
MFRC522 (optional)	miguelbalboa/MFRC522	RC522 RFID cards.
WiFiManager	tzapu/WiFiManager	First-boot Wi-Fi setup portal on the device.

Firmware Modules

net

Wi-Fi connection, reconnects, NTP time sync, TLS client with pinned CA.

rules

Loads the snapshot from LittleFS, looks up users, checks grants and schedules.

decide

Single function that turns a credential into granted or denied with a reason.

queue

Ring buffer of up to 5,000 events in flash, uploaded in batches of 50.

api

Signs and sends heartbeat, sync, events and command acknowledgements.

readers

QR scanner, keypad, fingerprint and RFID drivers, each producing a credential event.

door

Relay timing, door contact monitoring, forced-open and held-open alarms.

ui

OLED messages, LEDs, buzzer patterns, local enrolment menu.

Main Loop

```
void loop() {
  readers.poll();                          // QR, keypad, fingerprint, RFID
  if (Credential c; readers.next(c)) {
    Decision d = decide(c, rules, rtc.now());
    if (d.granted) door.unlock(rules.unlockSeconds);
    ui.show(d);                            // OLED, LED, buzzer
    queue.push(Event::access(c, d, rtc.now()));
  }
  door.monitor(queue);                     // forced open, held open, closed
  if (every(5000))  api.heartbeat();       // status up, commands down
  if (api.rulesChanged()) rules.sync();    // download new snapshot
  if (every(10000)) queue.flush(api);      // upload pending events
}
```

The Decision Logic

Every credential goes through the same checks, in this order. The first failed check denies access and becomes the reason recorded in the log.

#	Check	Deny reason if it fails
1	Credential is readable and well formed (QR signature valid, PIN hash matches, fingerprint matched a slot).	invalid_credential
2	Credential is not revoked and the user is active.	revoked / user_inactive
3	For QR: token has not expired, was issued for this site, and its ID has not been used already.	qr_expired / qr_replayed

#	Check	Deny reason if it fails
4	The user has a grant for this door.	no_grant
5	Today is between the grant's valid-from and valid-until dates.	grant_not_active
6	The current time falls inside the grant's schedule (for example weekdays 07:00 to 18:00).	outside_schedule
7	The door's policy is satisfied (for example PIN plus fingerprint on high-security doors).	second_factor_required
8	Too many failed attempts have not locked out the keypad.	locked_out

Offline Mode

- The rule snapshot is stored in flash and survives reboots and power cuts.
- Time comes from the DS3231 RTC, which is corrected from NTP whenever the device is online.
- Events are queued in flash and uploaded in order once the connection returns; nothing is lost.
- QR codes still work offline because the door verifies the signature with a public key it already holds.
- Changes made in the dashboard while a door is offline (new grants, revocations) take effect the moment it reconnects and syncs.
- The dashboard shows each device as online or offline using its last heartbeat time, and raises an alert after 60 seconds of silence.

Device Provisioning

- An administrator adds the door and device in the dashboard and receives a **device ID** and a one-time **device secret**.
- On first boot the controller opens a Wi-Fi setup hotspot (Starling-Setup-XXXX). The installer connects with a phone, chooses the site Wi-Fi, and pastes the device ID and secret.
- The controller stores them in NVS, connects, performs its first sync and appears online in the dashboard.
- For production, enable ESP32 flash encryption and secure boot so the secret and cached rules cannot be read from a stolen board.

SECTION 06

6. Credentials

Three ways in, each designed so that a copied or stolen value is of little use.

QR Code on the Phone

The portal shows a QR code on the user's phone that changes every 30 seconds. Each code is a compact token signed by the server with an ECDSA P-256 private key (ES256). The door holds only the matching public key, so it can check the signature without the internet.

```
QR payload (JWT, ES256)
{
  "sub": "8f1c...e2",          // user id
  "site": "hq",                // site the token is valid for
  "jti": "b71f9c0a",           // unique token id, door rejects repeats
  "iat": 1792570800,           // issued at (Unix time)
  "exp": 1792570860            // expires 60 seconds later
}
```

- Signed on the server by a Next.js route handler using the **jose** library; the private key is a server-only environment variable.
- Rendered in the browser with **qrcode.react**. The page refreshes the token automatically and works as an installable PWA on the phone.
- The door remembers recently used token IDs, so a screenshot or photo of a code cannot be replayed.
- If a phone is lost, the administrator suspends the user and every door rejects them after its next sync.

PIN

- Users choose a 6-digit PIN in the portal. The server stores only a PBKDF2-HMAC-SHA256 hash with a per-user salt.
- The hash and salt are included in the door's snapshot, and the ESP32 hashes the typed PIN the same way and compares.
- Five wrong attempts lock the keypad for five minutes and raise an alert.
- Short PINs are guessable, so on high-security doors the policy should be PIN plus fingerprint.

Fingerprint

- Enrolment: an officer selects the user and the door in the dashboard and clicks Enrol. The command reaches the door on the next heartbeat.
- The OLED asks the user to place the same finger several times. The R503 builds the template and stores it in a free slot (up to 200).

- The controller reports the slot number; the cloud stores only **user to slot** in **biometric_enrollments**. The template itself never leaves the sensor.
- At the door, the sensor searches its own memory and returns the matching slot; the firmware maps it to a user and continues with the normal checks.
- Each door's sensor has its own templates, so users enrol once per door (or per sensor). Deleting a user sends a command to remove their slots.
- Users must give consent for biometric enrolment, and PIN or QR must remain available as an alternative.

Face Recognition

Face recognition was requested. It can be done on an ESP32-S3 camera board (for example ESP32-S3-EYE running Espressif's ESP-WHO), but it is only suitable for a small number of users and has weak protection against photos. The recommendation is:

- Version 1: use QR, PIN and fingerprint as described above.
- Later: add a dedicated face recognition terminal with liveness detection at doors that need it, connected to the controller through its relay or Wiegand output, or through the terminal's own API.

RFID Cards (optional)

If an RC522 reader is fitted, a card's unique ID can be linked to a user in the dashboard. Basic Mifare card IDs can be cloned, so cards should only be used with a second factor or on low-risk doors.

Door Policies

Policy	Who it suits
Any one credential (QR, PIN or fingerprint)	General doors, main entrance during office hours.
QR or fingerprint	Doors where PIN sharing is a concern.
PIN plus fingerprint	Server rooms, stores, offices with sensitive material.
Locked down	Emergency lock-down: only administrators and remote unlock.

SECTION 07

7. Device API (Supabase Edge Functions)

A small HTTPS API used only by door controllers. It is written in TypeScript and runs as Supabase Edge Functions on Deno.

Request Signing

Every request carries three headers. The Edge Function looks up the device secret in Supabase Vault, recomputes the signature and rejects the request if it does not match or if the timestamp is more than five minutes old.

```
X-Device-Id:  dev_7d2a91
X-Timestamp:  1792570800
X-Signature:  hex( HMAC_SHA256( device_secret, timestamp + "." + body ) )
```

Endpoints

Endpoint	Sent by device	Returned to device
POST /device-heartbeat	Every 5 s: firmware version, IP, Wi-Fi signal, door state, queue length, rules version held.	Pending commands (unlock, enrol, delete, reboot, update) and the latest rules version.
POST /device-sync	When the rules version changed.	Full rule snapshot for this door (see below).
POST /device-events	Batches of up to 50 access, alarm and status events.	IDs of accepted events so they can be removed from the queue.
POST /device-command-ack	Result of each executed command.	OK.
POST /device-enroll-result	Fingerprint slot number or error for an enrolment.	OK.

Rule Snapshot Example

```
{
  "rules_version": 42,
  "door": { "id": "d_main", "policy": "any", "unlock_seconds": 5, "held_open_alarm_s": 30 },
  "qr_public_key": "-----BEGIN PUBLIC KEY----- ...",
  "timezone": "Africa/Blantyre",
  "users": [
    { "uid": "8f1c...e2", "active": true,
      "pin": { "hash": "9a4f...", "salt": "c1d2...", "iter": 10000 },
      "fp_slots": [12],
      "valid_from": "2026-10-22", "valid_until": "2027-10-21",
      "schedule": [ { "days": [1,2,3,4,5], "from": "07:00", "to": "18:00" } ] }
  ],
  "revoked_jti": ["b71f9c0a"]
}
```

Remote Unlock

- An officer clicks Unlock in the dashboard. A server action checks the role and inserts a row in **device_commands**.
- The door's next heartbeat (within 5 seconds) returns the command; the ESP32 pulses the relay and acknowledges.
- The command, who sent it and when it executed are recorded in the audit log and the access log.

SECTION 08

8. Web Application Modules

The Next.js application serves three audiences from one codebase, with separate areas and permissions for each.

Public and Accounts

- Landing page, sign up with email verification, login, password reset, optional Google sign in.
- Profile with name, phone and photo; administrators and officers are added by invitation.

Access Requests (the application)

- Request form: site, doors or door group, reason, start and end dates, ID document upload.
- Status tracking: pending, approved, rejected, cancelled, with email notifications.

My Access (member area)

- Live rotating QR code, installable as a phone app (PWA).
- Set or change PIN, see enrolled fingerprints, list of doors and hours allowed.
- Personal entry history.

Approvals and Grants (admin)

- Review requests, approve with doors, schedule and validity dates, or reject with a reason.
- Edit, extend, suspend or revoke grants at any time; changes reach doors within seconds.

Doors, Devices and Schedules (admin)

- Sites, doors, door groups, policies and unlock times.
- Device registry with online status, last seen, firmware version, signal strength and queue length.
- Register new devices, rotate secrets, push firmware updates.
- Schedules such as Office Hours, 24/7 or Weekends.

Live Monitoring (officer)

- Real-time event feed with photo of user, door, credential type and result.
- Alerts: door forced open, door held open, device offline, repeated failed PINs.
- Remote unlock and fingerprint enrolment commands.

Reports and Audit

- Entry reports by person, door and date range, CSV export.
- Audit log of every administrative action and remote unlock.

Users and Roles

Role	Can do
Member / Applicant	Request access, view own QR code, set PIN, see own history.
Security Officer	Watch live events and alerts, remote unlock, start fingerprint enrolment, view all logs.
Administrator	Approve requests, manage grants, doors, devices, schedules and users, run reports.
Super Admin	Everything above plus administrator accounts and system settings.

Route Map

Route	Access	Purpose
/, /login, /register	Public	Landing and authentication
/request-access	Member	Submit an access request
/my-access, /my-access/qr	Member	Doors allowed, live QR code, PIN, history
/monitor	Officer	Live events, alerts, remote unlock
/admin/requests	Admin	Review and approve requests
/admin/people	Admin	Users, roles, grants, credentials
/admin/doors, /admin/devices	Admin	Doors, policies, device registry, provisioning
/admin/schedules	Admin	Time schedules
/admin/reports, /admin/audit	Admin	Reports, exports, audit log
/api/qr-token	Member (server)	Issues a signed QR token

SECTION 09

9. Technology Stack and Tools

Everything needed to build, run and install the platform, grouped by layer. Versions are pinned at setup and should be the latest stable releases.

Web Application

Layer	Technology	Purpose
Framework	Next.js (App Router)	Pages, server components, server actions, route handlers.
Language	TypeScript (strict)	Type safety across the codebase.
UI	React 19, Tailwind CSS v4, shadcn/ui, Lucide icons	Accessible, consistent interface.
Forms	React Hook Form + Zod	Validation shared by browser and server.
Tables and charts	TanStack Table, Recharts	Logs, reports and dashboard charts.
QR	jose (signing), qrcode.react (display)	Signed rotating QR codes.
PWA	Next.js web app manifest + service worker	Install the member area on phones.
Email	Resend + React Email	Approvals, alerts and notifications.

Cloud Backend: Supabase

Service	Use in Starling
Postgres + Row Level Security	All data, with access rules enforced in the database.
Auth	User accounts, sessions, email verification, roles.
Edge Functions (Deno)	The device API: heartbeat, sync, events, acknowledgements.
Realtime	Live event feed and device status in the dashboard.
Storage	ID documents (private bucket) and firmware binaries.
Vault	Encrypted storage of device secrets.
Database functions and triggers	Rules version bump, audit logging, offline detection (pg_cron).

Hardware and Firmware Tools

Tool	Purpose
VS Code + PlatformIO	Build, flash and monitor ESP32 firmware.
Arduino IDE 2	Quick hardware tests of individual modules.
USB-C data cable, CP210x / CH340 drivers	Flashing and serial logs.
Espressif esptool	Erase flash, burn eFuses for flash encryption and secure boot.
Digital multimeter	Check 12 V and 5 V rails, continuity and relay contacts.
Soldering kit, crimp tool, ferrules, heat shrink	Reliable wiring connections.
Drill, wall plugs, cable glands, labels	Mounting and cable entry.
Breadboard and jumper wires	Bench test before mounting.

Development and Operations

Tool	Purpose
Node.js LTS and pnpm	Run and build the web app.
Supabase CLI + Docker	Local database, migrations, Edge Function development and deployment.
Git + GitHub	Source control for web, firmware and database code.
GitHub Actions	Lint, type check, tests and builds on every push.
Vercel	Hosting for the web application.
Bruno or Postman	Testing the device API by hand.
Sentry	Error tracking for the web app.
Vitest, Playwright, Deno test, PlatformIO Unity tests	Automated testing across web, API and firmware.

SECTION 10

10. Database Design

Supabase Postgres, UUID keys, timestamps on every row and Row Level Security on every table.

Enumerations

Enum	Values
user_role	member, officer, admin, super_admin
request_status	pending, approved, rejected, cancelled
credential_type	qr, pin, fingerprint, rfid, face
door_policy	any, qr_or_fingerprint, pin_and_fingerprint, locked_down
event_result	granted, denied, alarm, info
command_type	unlock, enroll_fingerprint, delete_fingerprint, sync, reboot, update_firmware
command_status	pending, sent, done, failed, expired

Tables

Table	Purpose	Key columns
profiles	One row per user.	id (auth.users), full_name, phone, avatar_url, role, is_active
sites	Buildings or locations.	id, name, address, timezone
doors	Each controlled door.	id, site_id, name, policy, unlock_seconds, held_open_alarm_s, rules_version
devices	ESP32 controllers.	id, door_id, serial, mac, secret_id (Vault), firmware_version, last_seen_at, ip, rssi, status
schedules	Time windows.	id, name, rules (jsonb: days, from, to)
access_requests	Applications for access.	id, user_id, site_id, door_ids, reason, start_date, end_date, document_path, status, reviewed_by, reviewed_at, note
access_grants	Approved permissions.	id, user_id, door_id, schedule_id, valid_from, valid_until, request_id, granted_by, revoked_at
credentials	PINs and card IDs.	id, user_id, type, pin_hash, pin_salt, rfid_uid, status
biometric_enrollments	Fingerprint slots.	id, user_id, device_id, sensor_slot, enrolled_at
device_commands	Commands to doors.	id, device_id, type, payload, status, created_by, created_at, executed_at
access_events	Every attempt and alarm.	id, device_id, door_id, user_id, credential_type, result, reason, occurred_at, received_at
alerts	Things needing attention.	id, type, door_id, device_id, message, acknowledged_by, acknowledged_at
audit_logs	Administrative actions.	id, actor_id, action, entity, entity_id, metadata, created_at

Triggers and Jobs

Name	Behaviour
handle_new_user()	Creates a profile with role member on sign-up.
bump_rules_version()	On any change to grants, credentials, schedules, enrolments or the door, increments doors.rules_version so devices re-sync.
log_audit()	Writes an audit_logs row for changes made by admins and officers.
raise_alerts()	Creates alerts for forced-open, held-open and repeated failed attempts from access_events.
mark_offline (pg_cron, every minute)	Marks devices offline after 60 seconds without a heartbeat and raises an alert.

SECTION 11

11. Security and Privacy

Cloud and Web

- Row Level Security on every table: members see only their own data, officers see events, admins manage everything.
- The Supabase secret key is used only inside Edge Functions and server-only code, never in the browser or on a device.
- Every server action re-checks the user's role and validates input with Zod.
- Administrators and officers must use multi-factor authentication.
- Daily backups with point-in-time recovery on the production database.

Devices

- Each device has its own secret, kept in Supabase Vault in the cloud and in encrypted NVS on the device.
- All traffic is HTTPS with the Supabase certificate authority pinned in the firmware.
- Signed requests with timestamps prevent forged or replayed device messages.
- Flash encryption and secure boot on production boards; firmware updates are only accepted over HTTPS from Storage.
- A device can be disabled, or its secret rotated, from the dashboard at any time.

Physical

- Controller, relay and power on the secure side; only readers outside.
- Door contact detects forced entry and doors held open.
- Fail-safe lock, exit button and break-glass for safe escape at all times.
- Battery backup keeps doors operating and logging during power cuts.

Personal and Biometric Data

- Fingerprint templates are stored only inside the sensor and are deleted when access is removed.
- ID documents are in a private bucket and opened only through short-lived signed links.
- Users give explicit consent to biometric enrolment and can always use QR or PIN instead.
- Access logs are kept for a defined period (for example 12 months) and then deleted automatically.

SECTION 12

12. Project Structure

One repository holds the web application, the Supabase project and the firmware.

```
starling-application-portal/
  web/                              # Next.js application
    src/app/
      (public)/  (auth)/            # landing, login, register
      (member)/my-access/           # QR, PIN, history
      (member)/request-access/
      (officer)/monitor/            # live events, remote unlock
      (admin)/admin/                # requests, people, doors, devices, schedules, reports
      api/qr-token/route.ts         # signs QR tokens
    src/features/                   # requests, grants, doors, devices, events ...
    src/lib/supabase/               # browser, server and admin clients
    src/types/database.types.ts     # generated
  supabase/
    migrations/                     # schema, RLS, triggers
    functions/
      device-heartbeat/  device-sync/  device-events/
      device-command-ack/  device-enroll-result/
      _shared/verify-device.ts      # HMAC check
    seed.sql
  firmware/door-controller/
    platformio.ini
    include/config.h                # pins, timings
    src/main.cpp
    src/net/  src/api/  src/rules/  src/readers/
    src/decide/  src/door/  src/queue/  src/ui/
    test/                           # Unity tests (decision logic)
  docs/                             # this document, wiring photos
  .github/workflows/ci.yml
```

SECTION 13

13. Setup Guide

Do these steps in order: cloud first, then the web application, then the firmware.

1. Supabase Project

```
pnpm dlx supabase login
pnpm dlx supabase link --project-ref <your-project-ref>
pnpm dlx supabase db push                    # create tables, RLS, triggers
pnpm dlx supabase functions deploy device-heartbeat device-sync \
  device-events device-command-ack device-enroll-result
pnpm dlx supabase secrets set QR_PUBLIC_KEY="$(cat qr_public.pem)"
```

2. QR Signing Keys (once)

```
openssl ecparam -name prime256v1 -genkey -noout -out qr_private.pem
openssl ec -in qr_private.pem -pubout -out qr_public.pem
# qr_private.pem  -> Vercel env QR_SIGNING_PRIVATE_KEY (never commit)
# qr_public.pem   -> sent to doors inside the rule snapshot
```

3. Web Application

```
cd web
pnpm install
cp .env.example .env.local        # fill in the values below
pnpm dev                          # http://localhost:3000
pnpm typecheck && pnpm lint && pnpm build
```

Variable	Scope	Description
NEXT_PUBLIC_SUPABASE_URL	Public	Supabase project URL.
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Public	Publishable (anon) key; RLS protects data.
SUPABASE_SECRET_KEY	Server only	Secret key; bypasses RLS. Never expose.
QR_SIGNING_PRIVATE_KEY	Server only	ES256 private key for QR tokens.
NEXT_PUBLIC_SITE_URL	Public	Base URL for links and redirects.
RESEND_API_KEY, EMAIL_FROM	Server only	Email sending.

Deploy by importing the repository into Vercel with root directory **web** and adding the same variables.

4. Firmware

```
; firmware/door-controller/platformio.ini
[env:esp32-s3]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200
board_build.filesystem = littlefs
lib_deps =
  bblanchon/ArduinoJson
  adafruit/Adafruit Fingerprint Sensor Library
  robtillaart/I2CKeyPad
  olikraus/U8g2
  adafruit/RTClib
  miguelbalboa/MFRC522
  tzapu/WiFiManager
```

```
cd firmware/door-controller
pio run                     # build
pio run -t upload           # flash over USB
pio device monitor          # watch serial logs
```

Set the API base URL (https://your-project.supabase.co/functions/v1) in **include/config.h**. The device ID and secret are entered through the setup hotspot, not compiled into the firmware.

SECTION 14

14. Installation Day Guide

A practical checklist for installing the first door tomorrow.

Before Going On Site

- Supabase project live: migrations pushed, Edge Functions deployed, secrets set.
- Web app deployed on Vercel; super admin account created; site, door and schedule added.
- Device registered in the dashboard; device ID and secret written down for the installer.
- Firmware flashed and **bench tested**: every reader, relay, buzzer and OLED working on the desk with a 12 V supply.
- One test member approved with QR, PIN and (after mounting) fingerprint.
- Tools and parts packed (see Hardware and Firmware Tools).

On Site

Step	Action
1	Check the door and frame; fit the magnetic lock and armature plate aligned and tight.
2	Mount the reader housing outside and the controller box inside; drill and run cables through.
3	Wire the lock power loop: PSU, fuse, break-glass NC, exit button NC, relay COM to NC, lock, with the diode across the lock.
4	Wire readers, door contact and exit button signal to the controller following the pin map.
5	Measure 12 V and 5 V with the multimeter before connecting the ESP32.
6	Power on, join the Starling-Setup hotspot, choose Wi-Fi, enter device ID and secret.
7	Confirm the device shows online in the dashboard and the rules version matches.
8	Enrol the test user's fingerprint from the dashboard.
9	Run every acceptance test below and record the results.
10	Close and lock the controller box; label cables; hand over to the site owner.

Acceptance Tests

Test	Expected result
Valid QR code during schedule	Door unlocks for 5 seconds, green LED, event granted in dashboard within seconds.
Screenshot of an old QR code	Denied: qr_expired.
Correct PIN / wrong PIN	Unlock / denied; five wrong PINs lock the keypad and raise an alert.
Enrolled fingerprint / unknown finger	Unlock / denied: invalid_credential.
User outside schedule hours	Denied: outside_schedule.
Grant revoked in dashboard	Denied at the door within about 10 seconds.
Remote unlock from dashboard	Door unlocks within 5 seconds; audit log entry created.
Exit button pressed	Door releases immediately even with the ESP32 unplugged.
Door forced open / held open 30 s	Alarm event and alert in the dashboard.
Wi-Fi switched off	QR, PIN and fingerprint still work; events upload after Wi-Fi returns.
Mains power cut	Battery keeps system running; with battery removed the lock releases (fail-safe).
Break-glass operated	Lock power cut, door releases.

SECTION 15

15. Testing and Quality

Level	Tool	What is tested
Static checks	TypeScript, ESLint	Type errors and unsafe patterns on every push.
Unit (web)	Vitest	Schemas, schedule logic, QR token creation.
Database	pgTAP via Supabase CLI	RLS: each role sees only what it should.
Device API	Deno test, Bruno	Signature checks, sync snapshots, event ingestion.
Firmware unit	PlatformIO Unity	Decision logic, schedule checks, PIN hashing, QR verification.
Hardware bench	Desk rig with all modules	Readers, relay timing, offline mode before every install.
End-to-end	Playwright + real door	Apply, approve, credential, enter, see log.

GitHub Actions runs lint, type check, web tests, database tests and a firmware build on every push. A failing check blocks deployment.

SECTION 16

16. Risks and Mitigations

Risk	Impact	Mitigation
Internet or Wi-Fi outage	High	Local decisions from cached rules, offline event queue, RTC clock, optional Ethernet.
Power cut	High	Battery-backed PSU; fail-safe lock releases if all power is lost.
People trapped during fire	High	Fail-safe lock, exit button and break-glass in the power loop, fire alarm release.
Tampering at the reader	High	Controller and relay on the secure side, tamper switch, door contact alarms.
QR screenshots or sharing	Medium	60-second signed tokens, replay protection, PIN plus fingerprint on sensitive doors.
Stolen controller board	Medium	Flash encryption, secure boot, disable device and rotate secret from dashboard.
Misconfigured RLS	High	Policy tests in CI and review before launch.
Biometric privacy complaints	Medium	Templates stay on sensor, consent, alternatives always available.
Clock drift breaks schedules or QR	Medium	DS3231 RTC plus NTP correction on every sync.
Face recognition accuracy on ESP32	Medium	Deferred to a dedicated terminal with liveness detection.

SECTION 17

17. Appendix

Glossary

Term	Meaning
ESP32-S3	A low-cost Wi-Fi microcontroller from Espressif used as the door controller.
Relay	An electrically operated switch that turns lock power on or off.
Fail-safe lock	A lock that releases when power is removed (magnetic locks).
Fail-secure lock	A lock that stays locked when power is removed (some electric strikes).
UART, I2C, SPI	Wiring standards the ESP32 uses to talk to readers and modules.
Rule snapshot	The list of users, credentials and schedules a door needs to decide on its own.
Heartbeat	A small message a device sends every few seconds to say it is alive and fetch commands.
HMAC	A signature made with a shared secret proving a message came from a genuine device.
ES256 / ECDSA	Public-key signature used for QR tokens; the door only needs the public key.
RLS	Row Level Security: database rules deciding which rows each user can access.
Edge Function	A serverless TypeScript function hosted by Supabase.
OTA	Over-the-air firmware update.
PWA	Progressive Web App: a website that can be installed on a phone like an app.

Useful References

Resource	Link
Next.js documentation	nextjs.org/docs
Supabase documentation	supabase.com/docs
Supabase Edge Functions	supabase.com/docs/guides/functions
ESP32-S3 technical reference	espressif.com (ESP32-S3 documentation)
Arduino core for ESP32	github.com/espressif/arduino-esp32
PlatformIO	docs.platformio.org
Adafruit Fingerprint library	github.com/adafruit/Adafruit-Fingerprint-Sensor-Library
jose (JWT signing)	github.com/panva/jose

Document History

Version	Date	Changes
1.0	21 October 2026	Initial proposal. Author: Mazwewoh John Brindi.
2.0	21 October 2026	Rewritten as an access control platform: door hardware, wiring, firmware, device API, credentials, installation guide and new diagrams.